

Mini Security Operations Center

Terraform • Ansible • Wazuh

Conception et déploiement automatisé d'une plateforme SOC pour la collecte, l'analyse et la supervision des événements de sécurité.

Objectif pédagogique

Maîtriser le provisionnement d'infrastructure avec Terraform et la configuration automatisée avec Ansible, à travers le déploiement d'un SIEM Wazuh opérationnel sur machines virtuelles VirtualBox.

FICHE TECHNIQUE

Technologies	Terraform, Ansible, Wazuh, VirtualBox, Ubuntu Server
Architecture	2 VMs : Serveur SOC + Machine Surveillée
Cas d'usage	Détection brute force SSH, modification /etc/passwd
Version	V1 — Architecture de base

Sommaire

Chapitre 1 — Contexte et Problématique	3
1.1 Contexte général	3
1.2 Problématique	3
Chapitre 2 — Objectifs du Projet	4
2.1 Objectif général	4
2.2 Objectifs spécifiques	4
Objectifs techniques	4
Objectifs d'apprentissage	4
Chapitre 3 — Choix Technologiques	5
Chapitre 4 — Architecture Générale	6
4.1 Vue d'ensemble (V1)	6
4.2 Flux réseau	6
4.3 Ressources allouées	6
Chapitre 5 — Description des Composants	8
5.1 Serveur SOC (VM-SOC)	8
Wazuh Manager	8
Wazuh Indexer	8
Wazuh Dashboard	8
5.2 Machine Surveillée (VM-CLIENT)	9
Wazuh Agent	9
Chapitre 6 — Fonctionnement Global	10
Chapitre 7 — Cas d'Usage de Détection	11
7.1 Cas 1 — Détection d'une attaque brute force SSH	11
7.2 Cas 2 — Création non autorisée d'un utilisateur	11
7.3 Cas 3 — Modification d'un fichier sensible	12
Chapitre 8 — Déploiement avec Terraform	13

8.1 Rôle de Terraform dans le projet	13
8.2 Provider utilisé	13
8.3 Variables d'infrastructure	13
8.4 Création des machines	14
8.5 Outputs pour Ansible	14
8.6 Cycle de vie Terraform	15
Chapitre 9 — Configuration avec Ansible	16
9.1 Rôle d'Ansible dans le projet	16
9.2 Inventaire des machines	16
9.3 Playbook principal	16
9.4 Rôles utilisés	17
9.5 Configuration de l'agent	17
9.6 Cycle de vie Ansible	18
Chapitre 10 — Structure du Projet	19
Chapitre 11 — Procédure de Déploiement	20
11.1 Prérequis	20
11.2 Étapes de déploiement	20
11.3 Vérification du fonctionnement	20
11.4 Déploiement complet en une commande	21
Chapitre 12 — Évolutions Futures	22
Chapitre 13 — Compétences Acquises	23

Chapitre 1 — Contexte et Problématique

1.1 Contexte général

La transformation numérique des entreprises s'accompagne d'une augmentation constante des cybermenaces. Les attaques par force brute, les accès non autorisés, les malwares et les tentatives d'escalade de privilèges sont devenus des risques quotidiens pour toute organisation connectée à Internet. Les vecteurs d'attaque se multiplient avec la diversification des environnements informatiques (postes utilisateurs, serveurs, conteneurs, services cloud), ce qui rend la supervision manuelle de plus en plus insuffisante.

Dans de nombreuses PME, les événements de sécurité restent dispersés dans différents fichiers journaux (logs) et ne sont ni centralisés, ni surveillés en temps réel. Cette dispersion rend la détection des incidents lente, augmente le temps de réaction en cas d'attaque et complique considérablement les investigations post-incident. Les équipes système et réseau se retrouvent souvent démunies face à un volume de logs qu'elles ne peuvent ni corrélérer ni analyser efficacement.

Face à ce problème, les Security Operations Centers (SOC) permettent de centraliser, analyser et surveiller en continu les événements de sécurité provenant de plusieurs systèmes. Un SOC s'appuie typiquement sur un outil SIEM (Security Information and Event Management) qui agrège les journaux, applique des règles de corrélation et émet des alertes en cas de comportement anormal. L'objectif de ce projet est de concevoir une version simplifiée d'un SOC moderne en utilisant les principes du DevOps et de l'Infrastructure as Code (IaC).

1.2 Problématique

Le projet vise à répondre à la question suivante : comment mettre en place une infrastructure de supervision de sécurité capable de collecter automatiquement les événements, de centraliser les journaux provenant de plusieurs machines, de détecter les comportements suspects, de générer des alertes pertinentes, et d'être entièrement reproductible grâce à l'automatisation ?

La réponse à cette problématique passe par l'utilisation conjointe de deux outils complémentaires : Terraform pour le provisionnement de l'infrastructure (création des machines virtuelles et du réseau) et Ansible pour la configuration logicielle des machines (installation des services Wazuh et déclaration des règles). Cette approche permet de déployer l'ensemble du SOC en quelques commandes, et de le reconstruire à l'identique sur n'importe quel environnement.

Chapitre 2 — Objectifs du Projet

2.1 Objectif général

L'objectif principal de ce projet est de concevoir une plateforme SOC automatisée permettant la collecte, l'analyse et la visualisation des événements de sécurité à l'aide de Wazuh, Terraform et Ansible. Au-delà du déploiement fonctionnel, le projet poursuit une finalité pédagogique : maîtriser concrètement l'utilisation de Terraform pour le provisionnement d'infrastructure, et d'Ansible pour la configuration automatisée des serveurs.

2.2 Objectifs spécifiques

Les objectifs spécifiques se déclinent en deux familles : d'une part les objectifs techniques liés au SOC, d'autre part les objectifs d'apprentissage liés aux outils DevOps.

Objectifs techniques

- Mettre en œuvre une infrastructure virtualisée sous VirtualBox avec Ubuntu Server.
- Automatiser la création des machines virtuelles avec Terraform (VM-SOC et VM-CLIENT).
- Automatiser la configuration des systèmes avec Ansible (installation et démarrage des services).
- Déployer une plateforme SIEM basée sur Wazuh (Manager, Indexer, Dashboard).
- Collecter les journaux système de plusieurs machines via l'agent Wazuh.
- Détecter automatiquement certains comportements malveillants (brute force, modification de fichiers sensibles).
- Produire des tableaux de bord de supervision accessibles depuis le navigateur.

Objectifs d'apprentissage

- Maîtriser le cycle de vie Terraform : init, plan, apply, destroy.
- Savoir structurer un projet Terraform (fichiers main.tf, variables.tf, outputs.tf).
- Maîtriser le cycle Ansible : inventaire, playbooks, rôles, modules.
- Comprendre le chaînage Terraform → Ansible (génération d'inventaire dynamique).
- Adopter les bonnes pratiques DevOps : versioning Git, documentation, reproductibilité.

Chapitre 3 — Choix Technologiques

Le choix des technologies a été guidé par trois critères : la maturité de l'outil, sa gratuité (open source) et sa pertinence pédagogique. L'ensemble de la pile logicielle peut être exécuté sur un poste de travail standard, sans dépendance cloud payante.

Technologie	Rôle dans le projet
VirtualBox	Hyperviseur local utilisé pour virtualiser les machines du SOC.
Ubuntu Server 22.04 LTS	Système d'exploitation des machines virtuelles, choix standard pour serveurs Linux stables.
Terraform	Outil d'Infrastructure as Code : provisionne les VMs et le réseau via le provider VirtualBox.
Ansible	Outil de Configuration as Code : installe et configure Wazuh sur les VMs via SSH.
Wazuh	Plateforme SIEM open source : détection d'intrusion, collecte de logs et corrélation d'événements.
Wazuh Indexer	Moteur de stockage et d'indexation des événements (basé sur OpenSearch).
Wazuh Dashboard	Interface web de visualisation des alertes et tableaux de bord.
Git	Gestion de version du code Terraform et Ansible (suivi des évolutions, collaboration).

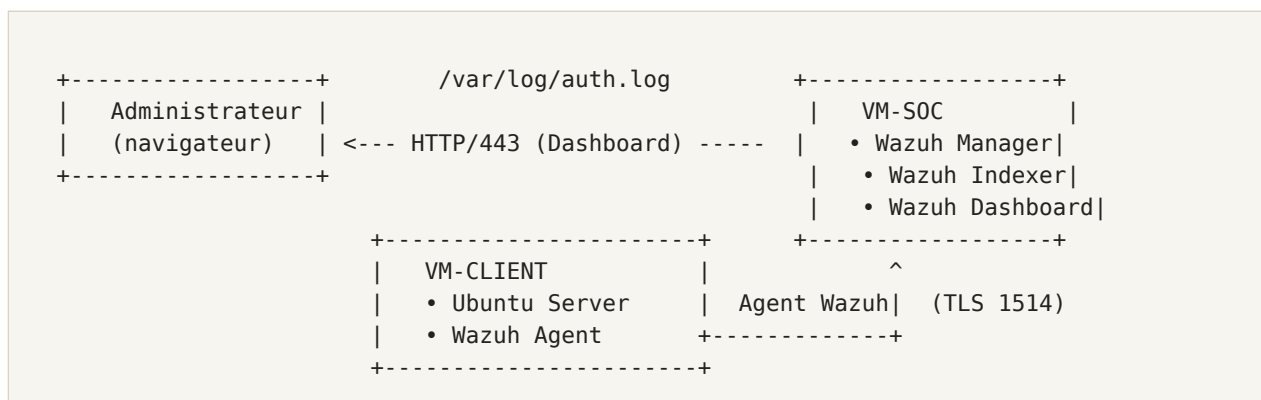
Wazuh a été retenu comme SIEM car il offre une suite complète et intégrée (Manager + Indexer + Dashboard) déployable sur une seule machine pour la V1. Sa compatibilité native avec Ansible (rôles officiels maintenus sur Ansible Galaxy) en fait un excellent support pour apprendre à automatiser le déploiement d'une stack réaliste de cybersécurité.

Chapitre 4 — Architecture Générale

4.1 Vue d'ensemble (V1)

L'architecture V1 est volontairement minimaliste : elle met en œuvre deux machines virtuelles only. La première, appelée VM-SOC, héberge l'ensemble des composants Wazuh (Manager, Indexer et Dashboard). La seconde, appelée VM-CLIENT, représente un serveur de production ou un poste utilisateur à surveiller et exécute uniquement l'agent Wazuh. Cette séparation permet de comprendre la logique client/serveur d'un SIEM tout en limitant la consommation de ressources sur le poste hôte.

Figure 1 — Schéma d'architecture V1



4.2 Flux réseau

Les deux machines virtuelles sont placées dans un réseau host-only VirtualBox, ce qui leur permet de communiquer entre elles et avec le poste hôte sans exposer directement les services à Internet. Trois flux principaux circulent sur ce réseau :

- Agent → Manager (port 1514/UDP) : transmission des logs collectés par l'agent Wazuh.
- Manager → Indexer (port 9200/TCP) : indexation des alertes générées par le Manager.
- Hôte → Dashboard (port 443/TCP) : accès à l'interface web depuis le navigateur de l'administrateur.

4.3 Ressources allouées

Les ressources sont dimensionnées au minimum requis par Wazuh. Le serveur SOC est plus gourmand car il héberge l'Indexer (OpenSearch) qui demande au moins 4 Go de RAM pour démarrer correctement. Le client reste léger car l'agent Wazuh consomme peu de ressources.

Machine	vCPU	RAM	Disque	Adresse IP
VM-SOC	2	6 Go	30 Go	192.168.56.10
VM-CLIENT	1	1 Go	15 Go	192.168.56.11

Chapitre 5 — Description des Composants

5.1 Serveur SOC (VM-SOC)

La machine VM-SOC est le cœur opérationnel du système. Elle regroupe les trois composants de la suite Wazuh, qui communiquent entre eux via des connexions chiffrées TLS. L'installation des trois services sur une seule machine simplifie le déploiement en V1 ; elle pourra être éclatée sur plusieurs VMs dans les versions ultérieures pour des raisons de performances.

Wazuh Manager

Le Manager est le moteur analytique du SIEM. Il reçoit les événements envoyés par les agents, les confronte à un ensemble de règles de détection (plus de 4000 règles par défaut) et génère des alertes lorsqu'un motif suspect est identifié. Il est responsable de quatre tâches principales :

- Recevoir les événements envoyés par les agents Wazuh sur le port 1514.
- Analyser les logs reçus et appliquer les règles de détection.
- Générer des alertes lorsqu'un comportement suspect est identifié.
- Transmettre les alertes au Wazuh Indexer pour stockage et recherche.

Wazuh Indexer

L'Indexer est un moteur de recherche et de stockage basé sur OpenSearch. Il reçoit les alertes produites par le Manager et les stocke sous forme indexée pour permettre des recherches rapides même sur de gros volumes de données. Il assure trois fonctions principales :

- Indexation temps réel des alertes générées par le Manager.
- Recherche full-text rapide sur l'historique des événements.
- Historisation des alertes pour les investigations post-incident.

Wazuh Dashboard

Le Dashboard est l'interface graphique web (accessible sur le port 443) qui permet à l'administrateur de superviser le SOC. Il interroge l'Indexer pour afficher les données sous forme de tableaux de bord interactifs. Ses principales fonctionnalités sont :

- Visualisation en temps réel des alertes de sécurité.
- Création de tableaux de bord personnalisés (top attaquants, géolocalisation, etc.).
- Investigation détaillée des incidents via recherche full-text.
- Gestion des agents Wazuh (statut, version, configuration).

5.2 Machine Surveillée (VM-CLIENT)

La machine VM-CLIENT représente un serveur de production ou un poste utilisateur dont on souhaite surveiller l'activité. Elle n'héberge qu'un seul composant : l'agent Wazuh. Cet agent est léger (consomme moins de 50 Mo de RAM), fonctionne en arrière-plan et communique avec le Manager de manière chiffrée.

Wazuh Agent

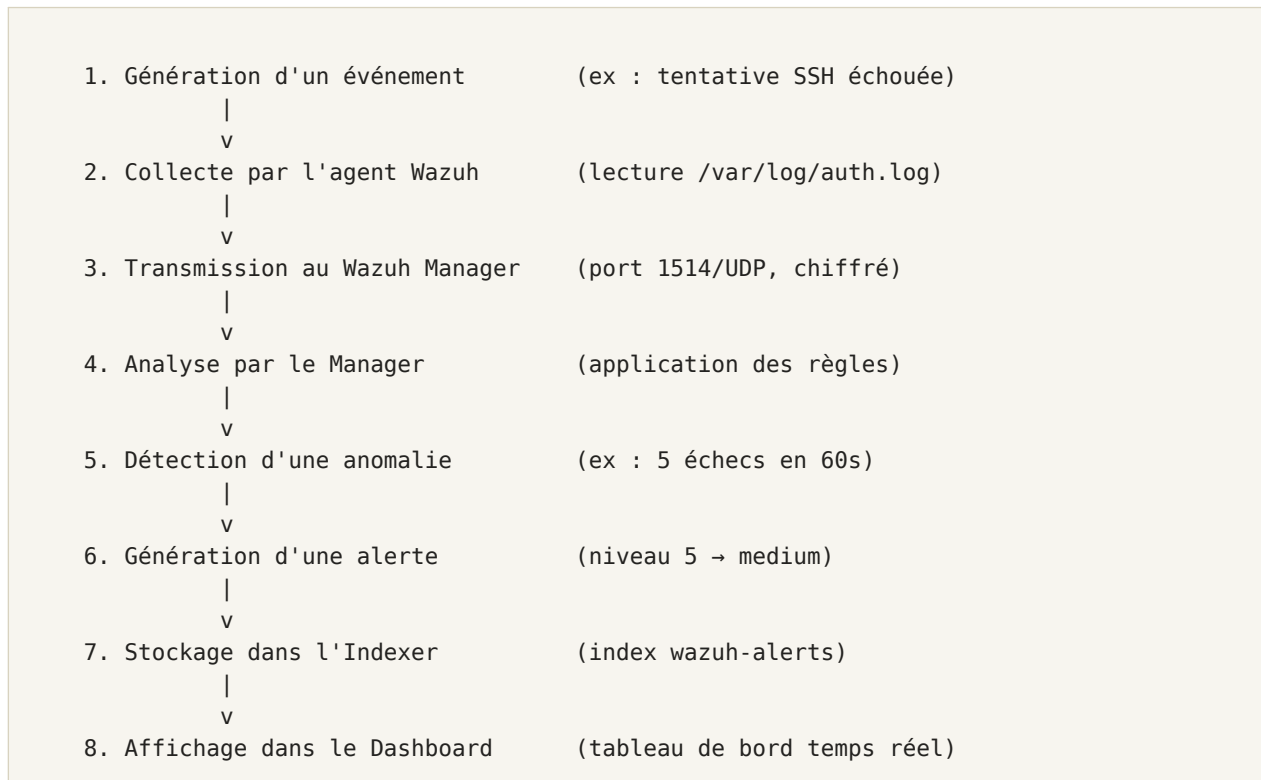
L'agent est installé en tant que service système (systemd) et démarre automatiquement au boot de la machine. Il collecte les logs locaux et les transmet au Manager. Ses responsabilités sont :

- Surveiller les fichiers système (notamment /etc/passwd, /etc/shadow, /etc/sudoers).
- Collecter les logs d'authentification (/var/log/auth.log).
- Détecter les modifications de fichiers sensibles (File Integrity Monitoring).
- Transmettre les événements au Wazuh Manager sur le port 1514.

Chapitre 6 — Fonctionnement Global

Le processus de surveillance suit une chaîne d'étapes bien définie, depuis la génération d'un événement sur la machine surveillée jusqu'à son affichage dans le tableau de bord Wazuh. Cette chaîne est entièrement automatique une fois les agents déployés et ne requiert aucune intervention manuelle en fonctionnement nominal.

Figure 2 — Chaîne de traitement d'un événement



À chaque étape, le système ajoute de la valeur : l'agent collecte et normalise, le Manager analyse et corrèle, l'Indexer stocke et indexe, et le Dashboard présente. Cette séparation des responsabilités est typique des architectures SIEM modernes et permet de scaler chaque composant indépendamment en production.

Chapitre 7 — Cas d'Usage de Détection

Trois cas d'usage ont été retenus pour valider le fonctionnement du SOC. Ils couvrent les principales catégories de menaces auxquelles un serveur Linux est exposé en environnement réel. Tous sont détectés nativement par les règles Wazuh par défaut, sans nécessiter de configuration avancée.

7.1 Cas 1 — Détection d'une attaque brute force SSH

Un attaquant tente de se connecter en SSH sur VM-CLIENT en essayant de nombreux mots de passe. Chaque tentative échouée génère une ligne Failed password dans le fichier /var/log/auth.log. L'agent Wazuh collecte ces lignes et les envoie au Manager, qui détecte la répétition via la règle 5712 et génère une alerte de niveau moyen.

Commande simulée sur VM-CLIENT

```
$ ssh root@192.168.56.11
Permission denied, please try again.
$ ssh root@192.168.56.11
Permission denied, please try again.
...

# Log généré dans /var/log/auth.log :
Feb 10 14:23:11 client sshd[2310]: Failed password for root from 192.168.56.1
Feb 10 14:23:13 client sshd[2310]: Failed password for root from 192.168.56.1
Feb 10 14:23:15 client sshd[2310]: Failed password for root from 192.168.56.1
```

Action du SOC : collecte des logs → détection de la répétition → génération d'une alerte de sécurité → affichage dans le tableau de bord avec niveau, source et heure de l'attaque.

7.2 Cas 2 — Création non autorisée d'un utilisateur

Un utilisateur malveillant (ou un attaquant ayant déjà compromis le système) crée un nouveau compte pour conserver un accès. La commande useradd génère une entrée dans le log /var/log/auth.log, qui est surveillée par Wazuh via la règle 5902 (compte ajouté).

Commande simulée sur VM-CLIENT

```
$ sudo useradd hacker

# Log généré :
Feb 10 14:25:42 client useradd[2401]: new user: name=hacker, UID=1001, GID=1001
```

Action du SOC : détection de la modification → journalisation de l'événement → notification de l'administrateur via le Dashboard.

7.3 Cas 3 — Modification d'un fichier sensible

Un attaquant tente de modifier le fichier `/etc/passwd` pour élever ses privilèges (par exemple en ajoutant un compte UID 0). Wazuh intègre un module de File Integrity Monitoring (FIM) qui surveille en continu les fichiers critiques et déclenche une alerte dès qu'un changement est détecté, avec comparaison entre l'état précédent et le nouvel état.

Commande simulée sur VM-CLIENT

```
$ sudo vim /etc/passwd
# (ajout manuel d'une ligne ou modification d'un UID)

# Alerte générée par Wazuh FIM :
Rule: 550 (File modified)
File: /etc/passwd
Mode: scheduled
Changes: size, md5, sha1
```

Chapitre 8 — Déploiement avec Terraform

8.1 Rôle de Terraform dans le projet

Terraform est responsable du provisionnement de l'infrastructure : il crée les machines virtuelles VM-SOC et VM-CLIENT dans VirtualBox, configure leur carte réseau, leurs ressources (CPU, RAM, disque) et injecte une clé SSH publique via cloud-init pour que Ansible puisse ensuite s'y connecter. Toute l'infrastructure est déclarée dans des fichiers .tf versionnés avec Git, ce qui garantit sa reproductibilité.

8.2 Provider utilisé

Le provider VirtualBox utilisé est terra-farm/virtualbox, un provider communautaire maintenu sur le GitHub de terra-farm. Il permet de piloter VirtualBox via son API en ligne de commande (VBoxManage). Bien que non officiel HashiCorp, il est largement utilisé dans les contextes pédagogiques et les environnements de test locaux.

Fichier : terraform/versions.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    virtualbox = {
      source = "terra-farm/virtualbox"
      version = "~> 0.2.0"
    }
  }
}

provider "virtualbox" {}
```

8.3 Variables d'infrastructure

Les paramètres variables (nom des VMs, adresses IP, ressources, chemin de l'image ISO Ubuntu) sont externalisés dans un fichier variables.tf et valorisés dans terraform.tfvars. Cette séparation permet de changer facilement la configuration sans modifier le code principal.

Fichier : terraform/variables.tf

```
variable "vm_soc_name"      { default = "vm-soc" }
variable "vm_client_name"  { default = "vm-client" }
variable "vm_soc_ip"       { default = "192.168.56.10" }
variable "vm_client_ip"    { default = "192.168.56.11" }
variable "ubuntu_iso_url"  { type = string }
variable "ssh_public_key"  { type = string }
variable "ssh_private_key_path" { type = string }
```

8.4 Création des machines

Le fichier `main.tf` déclare les deux ressources `virtualbox_vm` correspondant aux deux VMs. Chaque VM reçoit son image Ubuntu, ses ressources matérielles et un `user-data` `cloud-init` qui installe la clé SSH publique pour l'utilisateur `ansible`.

Fichier : `terraform/main.tf` (extrait)

```
resource "virtualbox_vm" "soc" {
  name     = var.vm_soc_name
  image    = var.ubuntu_iso_url
  cpus     = 2
  memory   = "6144mb"
  user_data = templatefile("cloud-init.tpl", {
    ssh_key = var.ssh_public_key
  })
  network_adapter {
    host_interface = "vboxnet0"
    type           = "hostonly"
  }
}

resource "virtualbox_vm" "client" {
  name     = var.vm_client_name
  image    = var.ubuntu_iso_url
  cpus     = 1
  memory   = "1024mb"
  user_data = templatefile("cloud-init.tpl", {
    ssh_key = var.ssh_public_key
  })
  network_adapter {
    host_interface = "vboxnet0"
    type           = "hostonly"
  }
}
```

8.5 Outputs pour Ansible

Terraform expose les adresses IP des VMs via le bloc `outputs.tf`. Ces valeurs sont utilisées pour générer dynamiquement le fichier d'inventaire Ansible après le `terraform apply`, garantissant la cohérence entre l'infrastructure provisionnée et l'inventaire utilisé par Ansible.

Fichier : `terraform/outputs.tf`

```
output "soc_ip" {
  value = virtualbox_vm.soc.network_adapter[0].ipv4_address
}

output "client_ip" {
  value = virtualbox_vm.client.network_adapter[0].ipv4_address
}
```

8.6 Cycle de vie Terraform

Le cycle de vie standard de Terraform comporte quatre commandes à connaître et maîtriser. Chacune joue un rôle précis dans la gestion de l'état de l'infrastructure.

Commande	Rôle
terraform init	Initialise le répertoire de travail et télécharge les providers nécessaires.
terraform plan	Affiche un plan d'exécution détaillé (ce qui sera créé, modifié, détruit).
terraform apply	Applique les changements et provisionne réellement l'infrastructure.
terraform destroy	Détruit toutes les ressources déclarées (utile pour repartir de zéro).

Chapitre 9 — Configuration avec Ansible

9.1 Rôle d'Ansible dans le projet

Une fois les VMs provisionnées par Terraform, Ansible prend le relais pour installer et configurer la suite Wazuh. Ansible fonctionne en agentless : il se connecte aux VMs via SSH et exécute des modules déclaratifs (apt, systemd, copy, template, etc.) décrits dans des playbooks YAML. Cette approche idempotente garantit que rejouer un playbook ne casse pas un système déjà configuré.

9.2 Inventaire des machines

Le fichier `inventory.ini` décrit les machines à configurer et les regroupe par rôle. Il est généré dynamiquement à partir des outputs Terraform, ce qui évite toute incohérence entre l'infrastructure réelle et l'inventaire Ansible.

Fichier : `ansible/inventory.ini`

```
[soc]
vm-soc ansible_host=192.168.56.10 ansible_user=ansible

[clients]
vm-client ansible_host=192.168.56.11 ansible_user=ansible

[all:vars]
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_python_interpreter=/usr/bin/python3
```

9.3 Playbook principal

Le playbook `site.yml` orchestre l'ensemble du déploiement. Il applique les rôles `wazuh-manager`, `wazuh-indexer` et `wazuh-dashboard` sur le serveur SOC, et le rôle `wazuh-agent` sur la machine cliente. Cette structure permet de séparer proprement les responsabilités et de réutiliser les rôles dans d'autres projets.

Fichier : `ansible/playbooks/site.yml`

```
---
- name: Déploiement du serveur SOC
  hosts: soc
  become: yes
  roles:
    - wazuh-indexer
    - wazuh-manager
    - wazuh-dashboard

- name: Déploiement de l'agent sur le client
  hosts: clients
  become: yes
  roles:
    - wazuh-agent
```

9.4 Rôles utilisés

Pour ne pas réinventer la roue, le projet s'appuie sur les rôles officiels maintenus par Wazuh sur Ansible Galaxy. Le fichier `requirements.yml` déclare les rôles à installer via la commande `ansible-galaxy install -r requirements.yml`. Cette pratique est une bonne habitude DevOps : réutiliser du code éprouvé plutôt que de tout écrire from scratch.

Fichier : `ansible/requirements.yml`

```
---
roles:
  - src: wazuh.wazuh_indexer
  - src: wazuh.wazuh_manager
  - src: wazuh.wazuh_dashboard
  - src: wazuh.wazuh_agent
```

9.5 Configuration de l'agent

Le rôle `wazuh-agent` installe le paquet et configure le fichier `/var/ossec/etc/ossec.conf` pour pointer vers le Wazuh Manager. Une fois démarré, l'agent s'enregistre automatiquement auprès du Manager et commence à transmettre les logs collectés.

Extrait de configuration : `/var/ossec/etc/ossec.conf`

```
<client>
  <server>
    <address>192.168.56.10</address>
    <port>1514</port>
    <protocol>tcp</protocol>
  </server>
  <enqueue_events>yes</enqueue_events>
</client>
```

9.6 Cycle de vie Ansible

Le cycle Ansible se résume à trois commandes essentielles, à exécuter après le terraform apply. La maîtrise de ces commandes est un objectif pédagogique central du projet.

Commande	Rôle
ansible-galaxy install -r requirements.yml	Télécharge les rôles Wazuh officiels depuis Ansible Galaxy.
ansible-playbook playbooks/site.yml	Exécute le playbook principal sur toutes les machines de l'inventaire.
ansible all -m ping	Vérifie la connectivité SSH vers toutes les machines avant le déploiement.

Chapitre 10 — Structure du Projet

Le projet suit une structure standard pour un dépôt DevOps : séparation claire entre le code Terraform (provisionnement) et le code Ansible (configuration), accompagnée d'un répertoire docs/ pour la documentation et d'un README à la racine. Cette organisation facilite la maintenance et la collaboration.

Arborescence du dépôt

```

soc-project/
├── terraform/
│   ├── versions.tf          # Versions Terraform et providers
│   ├── variables.tf         # Variables d'infrastructure
│   ├── main.tf              # Déclaration des VMs et du réseau
│   ├── outputs.tf           # Outputs (IPs) pour Ansible
│   ├── terraform.tfvars     # Valeurs des variables (hors Git)
│   └── cloud-init.tpl       # Template cloud-init (clé SSH)
├── ansible/
│   ├── ansible.cfg          # Configuration Ansible
│   ├── inventory.ini        # Inventaire (généré depuis Terraform)
│   ├── requirements.yml     # Rôles Galaxy à installer
│   ├── group_vars/
│   │   ├── soc.yml          # Variables spécifiques au SOC
│   │   └── clients.yml      # Variables spécifiques aux clients
│   └── playbooks/
│       ├── site.yml         # Playbook principal (orchestrateur)
│       ├── soc.yml          # Déploiement du serveur SOC
│       └── client.yml       # Déploiement des agents
├── docs/                    # Documentation, captures, schémas
├── .gitignore
└── README.md                # Procédure d'installation et d'utilisation
  
```

Le fichier .gitignore doit exclure le fichier d'état Terraform (terraform.tfstate), les sauvegardes d'état (*.tfstate.backup), le répertoire .terraform/ et le fichier terraform.tfvars qui peut contenir des secrets. Ces éléments ne doivent jamais être versionnés car ils peuvent contenir des informations sensibles (clés privées, mots de passe, adresses IP internes).

Chapitre 11 — Procédure de Déploiement

Cette section décrit la procédure complète pour déployer le SOC depuis un poste hôte vierge. Toutes les commandes sont à exécuter depuis la racine du dépôt Git `soc-project/`. La procédure suppose que VirtualBox, Terraform et Ansible sont déjà installés sur le poste hôte.

11.1 Prérequis

- VirtualBox 7.x avec l'Extension Pack installé.
- Terraform $\geq 1.5.0$ installé sur le poste hôte.
- Ansible ≥ 2.12 installé sur le poste hôte.
- Une paire de clés SSH ED25519 (`~/.ssh/id_ed25519`).
- 16 Go de RAM sur le poste hôte (recommandé pour faire tourner 2 VMs).
- L'image ISO Ubuntu Server 22.04 LTS téléchargée localement.

11.2 Étapes de déploiement

Éta pe	Commande	Description
1	<code>cd terraform/</code>	Se placer dans le répertoire Terraform.
2	<code>terraform init</code>	Initialiser Terraform et télécharger le provider VirtualBox.
3	<code>terraform plan</code>	Vérifier le plan d'exécution avant application.
4	<code>terraform apply</code>	Créer les deux VMs (validation requise).
5	<code>cd ../ansible/</code>	Se placer dans le répertoire Ansible.
6	<code>ansible-galaxy install -r requirements.yml</code>	Installer les rôles Wazuh officiels.
7	<code>ansible all -m ping</code>	Vérifier la connectivité SSH vers les VMs.
8	<code>ansible-playbook playbooks/site.yml</code>	Déployer Wazuh sur le SOC et l'agent sur le client.
9	Ouvrir <code>https://192.168.56.10</code>	Accéder au Dashboard Wazuh (admin / mot de passe généré).

11.3 Vérification du fonctionnement

Après le déploiement, plusieurs vérifications permettent de s'assurer que le SOC fonctionne correctement. Ces vérifications sont aussi l'occasion de manipuler les commandes Linux de diagnostic, ce qui participe à l'apprentissage.

- Côté SOC : vérifier que les trois services sont actifs avec `systemctl status wazuh-manager wazuh-indexer wazuh-dashboard`.
- Côté client : vérifier que l'agent est enregistré avec `systemctl status wazuh-agent`.
- Côté Dashboard : ouvrir <https://192.168.56.10> et vérifier que l'agent apparaît comme Active.
- Test d'alerte : déclencher 5 échecs SSH sur le client et observer l'alerte apparaître dans le Dashboard.

11.4 Déploiement complet en une commande

Pour faciliter l'usage, un simple Makefile peut enchaîner automatiquement les étapes Terraform puis Ansible. Cette automatisation est elle-même un bon exercice DevOps et permet de déployer l'ensemble du SOC en une seule commande.

Fichier : Makefile (extrait)

```
.PHONY: deploy destroy

deploy:
cd terraform && terraform init && terraform apply -auto-approve
./generate_inventory.sh
cd ansible && ansible-galaxy install -r requirements.yml
cd ansible && ansible-playbook playbooks/site.yml

destroy:
cd terraform && terraform destroy -auto-approve
```

Chapitre 12 — Évolutions Futures

Le projet V1 pose les fondations d'un SOC fonctionnel. Plusieurs évolutions sont envisageables pour enrichir la plateforme et monter en complexité. Cette feuille de route progressive est intéressante pédagogiquement car chaque version ajoute un niveau de difficulté maîtrisable.

Version	Évolution	Apport
V1 (actuelle)	2 VMs : SOC + 1 client Linux	Mise en place de la stack Wazuh de base.
V2	Ajout d'un 2 ^e client Linux	Démonstration de la centralisation multi-sources.
V3	Ajout d'une machine Windows	Surveillance d'un poste Windows via Wazuh Agent.
V4	Ajout d'un IDS réseau Suricata	Détection des attaques au niveau réseau (NIDS).
V5	Ajout de TheHive + Cortex	Gestion d'incidents et enrichissement automatique.
V6	SOC complet + Grafana	Supervision globale, métriques et tableaux de bord avancés.

Chaque version s'appuie sur la précédente et démontre une nouvelle compétence : V2 la scalabilité, V3 l'hétérogénéité des OS, V4 la détection réseau, V5 la réponse à incident, et V6 la supervision 360°. Toutes restent déployables via la même chaîne Terraform + Ansible, ce qui prouve la robustesse de l'approche IaC initiale.

Chapitre 13 — Compétences Acquises

À l'issue du projet, les compétences développées couvrent quatre domaines complémentaires : cybersécurité, administration système, réseau et DevOps. Ces compétences sont valorisables directement en entretien technique et constituent une base solide pour évoluer vers des métiers d'ingénieur DevOps, d'analyste SOC ou d'administrateur système.

Domaine	Compétences développées
Cybersécurité	SIEM (Wazuh), analyse de logs, détection d'intrusion, File Integrity Monitoring, investigation d'incidents via Dashboard.
Administration Système	Linux Ubuntu Server, services systemd, gestion des utilisateurs, sécurisation SSH (clés, désactivation root), lecture et analyse des logs.
Réseau	Architecture réseau host-only VirtualBox, communication inter-VM, gestion des ports (1514, 9200, 443), isolation réseau.
DevOps	Terraform (provisionnement IaC), Ansible (configuration as code), cycle init/plan/apply/destroy, inventaire et playbooks, versioning Git.
Gestion de projet	Documentation technique, versioning Git, structure de dépôt propre, reproductibilité des déploiements.

Au-delà des compétences techniques listées ci-dessus, le projet permet de développer des transversales essentielles : la capacité à lire et comprendre de la documentation officielle (Terraform Registry, Ansible Galaxy, docs Wazuh), le débogage méthodique d'une chaîne d'outils (Terraform → cloud-init → SSH → Ansible → Wazuh), et l'écriture de documentation technique claire destinée à un public technique. Ces compétences sont transversales à de nombreux métiers de l'IT et constituent un réel atout professionnel.